Tatkal Booking System

Design and Development of a Microservices Application

Scalable Services (SEZG583) - Assignment 1 - Design and Technical Report

Group member	BITS ID	Service owned
Dhruv Bhat	[BITS ID]	booking-service (saga orchestrator)
Ratani Snehi Rajesh Darshana	[BITS ID]	inventory-service
Atharva Kadam	[BITS ID]	payment-service and mock-bank-gateway
N Abhishek Reddy	[BITS ID]	notification-service, end-to-end tests

Source code: https://github.com/spaceman-dev/tatkal-booking-microservices
Demo video and documents: [Google Drive link - see section 10]

1. Problem and scope

1.1 Overview

Every Indian Railways train sets aside a small **Tatkal** quota per class that opens one day before the journey (10:00 for AC classes, 11:00 for the rest). Far more people want those seats than there are seats, so the first minutes after opening look like a stress test: thousands of attempts at once, sold-out pages that still showed seats a moment earlier, passengers retrying “Pay” when the bank page hangs, and the familiar “amount debited but ticket not booked - refund in 5-7 days”.

We built the back end of such a booking system. A passenger asks for 1-4 seats on a train, date and class. The system holds the seats, takes the payment through a bank, confirms the seats, issues a PNR and sends an SMS and e-mail. The interesting part is not the happy path but what happens when requests arrive together, arrive twice, or a step fails half way - which is what the assignment asks us to show.

The five required conditions map onto the domain naturally:

Condition	In our system
(a) 4+ services with distinct responsibilities	booking, inventory, payment, notification (plus a mock bank)
(b) a transaction spanning 3+ services	booking a ticket: inventory (hold), payment (charge, via the bank), inventory (confirm), notification
(c) a contended resource	the Tatkal seat quota of one train/date/class, e.g. the last 5 seats in Sleeper
(d) an operation unsafe to repeat	`POST /bookings` and `POST /payments` - they take seats and money
(e) a dependency that can be slow or fail	`mock-bank-gateway` (normal, slow, down, flaky, refunds-only outage) and a simulated SMS provider

1.2 Assumptions

- A booking is for one train, one date and one class, with 1-4 passengers. Every booking is a Tatkal booking; general quota, waiting list and RAC are not modelled.
- Seats are sold as a count per class. Berth numbers are allotted later (at charting) and are not part of this system.
- Each train/date/class has a base fare; the Tatkal charge follows the published Indian Railways rule (BR3). All money is stored as integer paise.
- The 10:00/11:00 opening time is not enforced, so the system can be demonstrated at any time.
- Payment is UPI or card through an external gateway that we do not control (simulated by our mock).

- A passenger is identified only by a mobile number; there are no accounts.

1.3 Business rules

Id	Rule	Owner
BR1	The seats of a train/date/class are never over-allocated. A request that does not fully fit is refused - no partial allocation.	inventory
BR2	A Tatkal booking has 1 to 4 passengers, each aged 5 or more (children under 5 travel without a berth).	booking
BR3	The fare is computed by the server: per-seat fare = base fare + Tatkal charge, the charge being 10 % (2S) or 30 % (other classes) of the base fare, clamped to the class minimum and maximum (for example Rs 300-400 in 3A). The client never sends a price.	inventory
BR4	A booking is charged at most once. Only a captured payment can be refunded, in full and only once.	payment
BR5	A booking moves only along its state machine (section 3.4).	booking
BR6	If the booking saga fails, everything that may have happened is undone, newest first: refund, then release the seats. A compensation that cannot run now is retried until it succeeds.	booking
BR7	Only a confirmed booking can be cancelled. Its seats return to the quota, but a confirmed Tatkal ticket gets no refund.	booking

Each rule is written once, as a plain function in the owning service's `domain/` package, and has unit tests; the full rule-to-code-to-test map is `docs/business-rules.md` (summarised in Appendix D).

1.4 Out of scope

Login and authorisation, any user interface, real payment and SMS providers (both are mocks we wrote), seat or berth selection, waiting list and charting, the booking-window rule, PNR enquiry for other passengers, and deployment (Docker images and Minikube are Assignment 2). Clients are our scripts, Swagger UI and curl.

2. Service decomposition

2.1 Responsibilities

Service	Responsibility in one sentence	Owns	Main operations
booking-service	Owns a passenger's booking from request to ticket (or failure) and orchestrates the steps needed to get there.	bookings, passengers, saga log, idempotency keys, outbox	place booking (saga), get, list, saga log, cancel
inventory-service	Owns trains, fares and the Tatkal quota, and is the only place seats can be held, confirmed or released.	trains, seat_inventory, seat_holds	availability, open quota, hold, confirm, release

Service	Responsibility in one sentence	Owns	Main operations
payment-service	Owns money movements for bookings and is the only service that talks to the bank.	payments, refunds, idempotency keys	charge, refund, look up
notification-service	Tells the passenger what happened and keeps a record of every message sent.	notifications	deliver event (SMS, e-mail), list
mock-bank-gateway	Stands in for the external UPI/card gateway (not a business service).	nothing persistent	charge, refund, look up, behaviour switch

2.2 Why these boundaries

We decomposed by **business capability** (CS05): selling seats, taking money, looking after a passenger's booking, and talking to the passenger are things a railway does separately and staffs separately. The common closure principle points the same way - each service changes for its own reason: a fare revision or a new class touches only inventory; a new bank or refund policy only payment; a new passenger rule only booking; a new SMS template or provider only notification.

The split also follows the data. The seat counter has exactly one writer (inventory) and money has exactly one writer (payment), so the two invariants we care most about - no overselling (BR1) and no double charge (BR4) - can each be enforced inside one local database transaction instead of across services. Booking does not own seats or money; it owns the *lifecycle* of a booking, which is why it coordinates the other two.

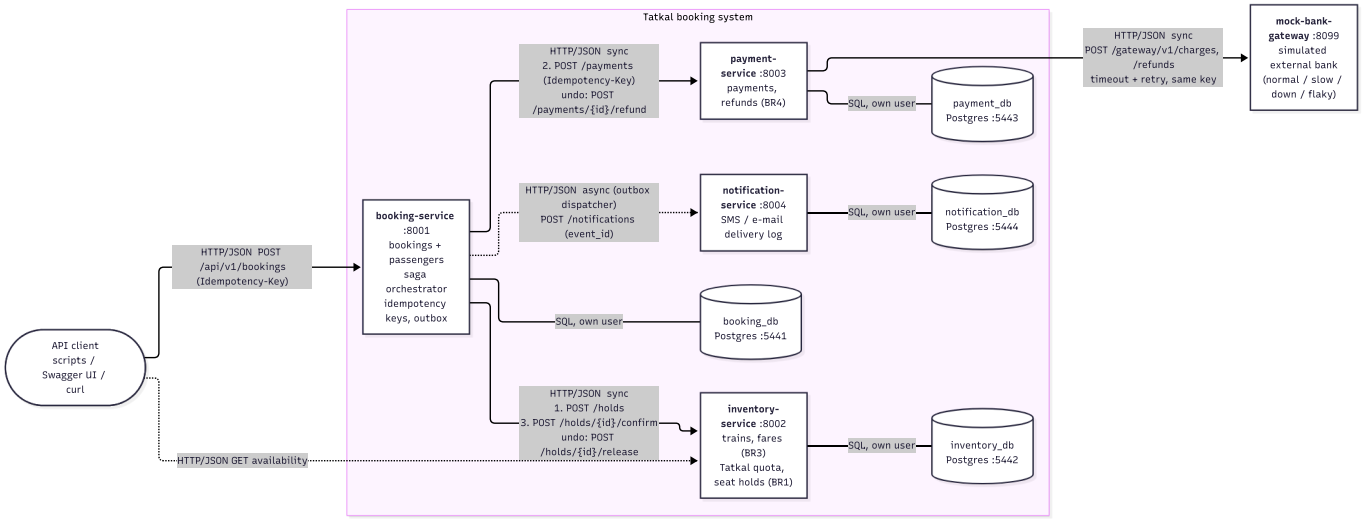
Finally the services behave differently under load, which matters for Assignment 2: inventory is the hot spot during a Tatkal rush, payment spends most of its time waiting for a slow bank, and notification can fall behind for minutes without anyone being harmed.

2.3 Alternatives we rejected

- **A separate train-catalogue service.** Trains and schedules look like their own subdomain, but every hold needs the fare and the quota together, and they change together. Splitting them would turn every hold into a two-service call - the "services that call each other on every request" smell the brief warns about.
- **One checkout service (booking + payment).** They are always used together, but payment has its own consistency rules (BR4), its own external dependency and its own failure modes. Keeping it separate isolates the slow bank and lets cancellations and compensations use the same refund operation.
- **Sending SMS from booking-service.** That would put a slow, failure-prone external call inside every booking request. A separate notification service fed asynchronously means a dead SMS provider cannot fail a booking.
- **A dedicated "saga orchestrator" service.** It would own no data and need to know every other service's API - a god service. The orchestration belongs next to the booking state it changes.
- **An API gateway.** With one entry point for writes (booking) and one read API (availability) it would add a component without adding behaviour in Assignment 1. It becomes a Kubernetes Ingress in Assignment 2.

3. Architecture

3.1 Components



Component diagram. Solid arrows are synchronous HTTP/JSON calls made while the passenger waits; dotted arrows are asynchronous or optional. Every service uses only its own database.

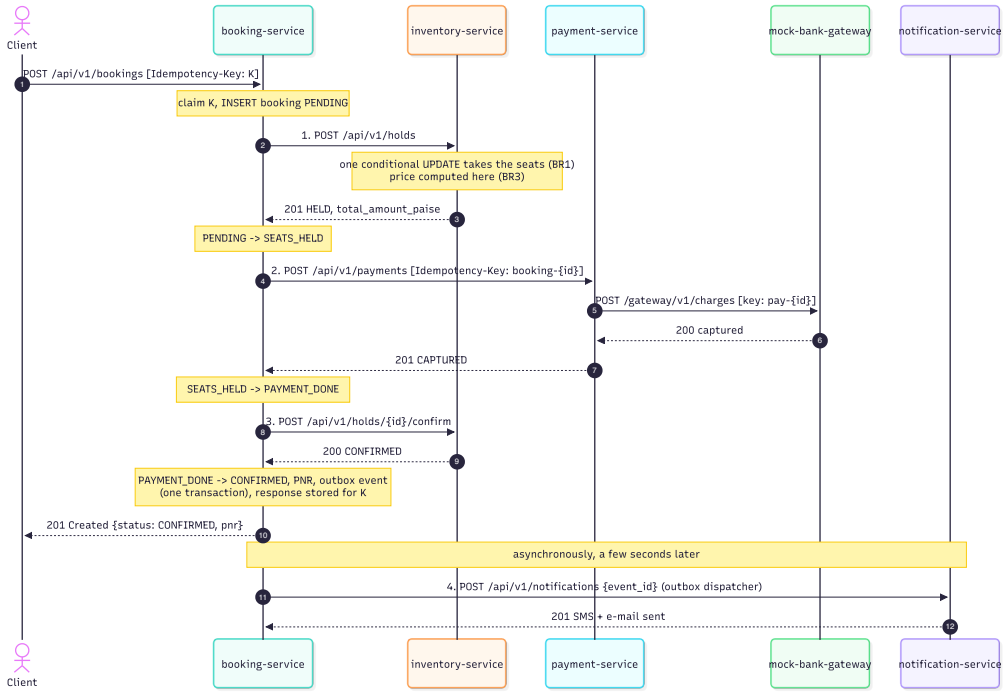
All services are Python 3.12 with FastAPI, talk HTTP/JSON, and each has its own PostgreSQL 16 database (four containers, four users). Each service is a self-contained folder with its own `requirements.txt`, `.env.example`, tests and README, and is started with `python -m app`.

The booking steps are **synchronous** because the passenger is waiting for a yes or no, and we need the price from step 1 before step 2. The longest chain is client -> booking -> payment -> bank, three hops, and the bank is behind payment on purpose so its slowness is handled in one place. Inventory and payment are called one after the other by booking (a fan-out, not a chain). Notifications are **asynchronous**: booking writes an event into an outbox table in the same transaction as the status change, and a background dispatcher delivers it.

Cross-cutting concerns are the same in every service:

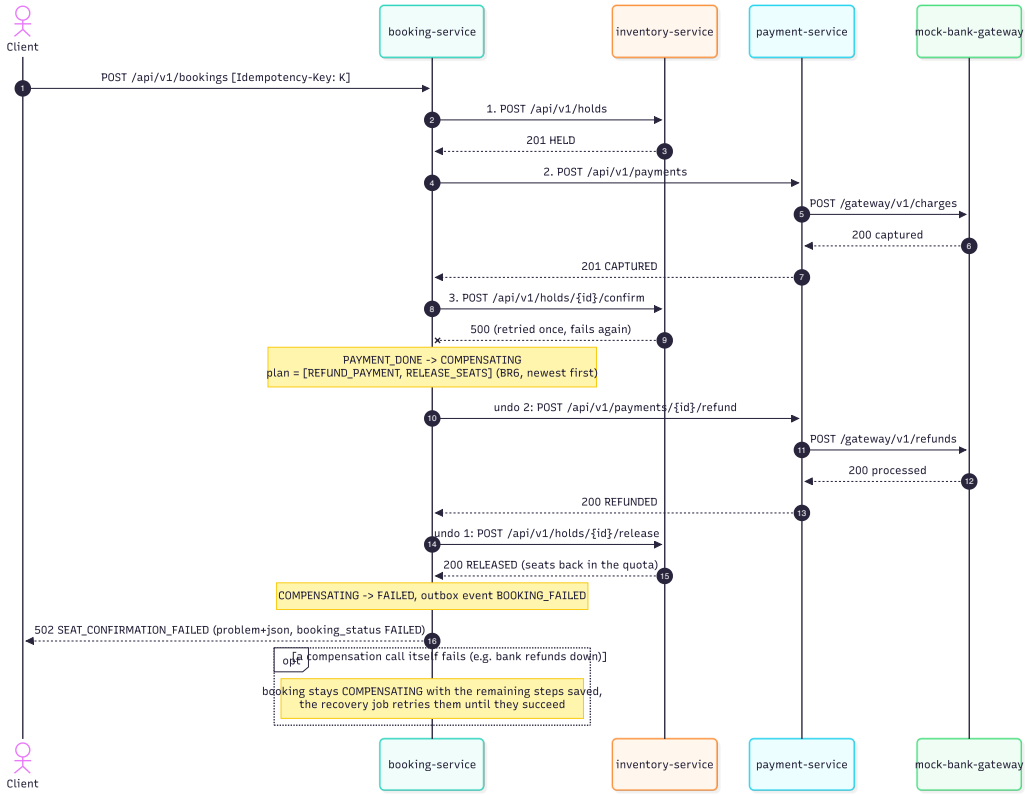
- **Configuration** only from environment variables (or a `.env` file); service URLs, ports and database URLs have no defaults, so a missing one stops the service at start-up.
- **Correlation id**: taken from the `X-Correlation-ID` header or generated, stored in a context variable, added to every log line, forwarded on every outgoing call and returned in the response. The outbox stores it with the event, so even the asynchronous SMS logs under the original request's id.
- **Logs** are one JSON object per line on stdout (`ts, level, service, correlation_id, msg, ...`).
- **`GET /health`** reports the service and checks its database (503 if the database is unreachable).

3.2 Main transaction - happy path



Booking a ticket when everything works. The passenger gets 201 with the PNR after step 3; the SMS and e-mail follow asynchronously.

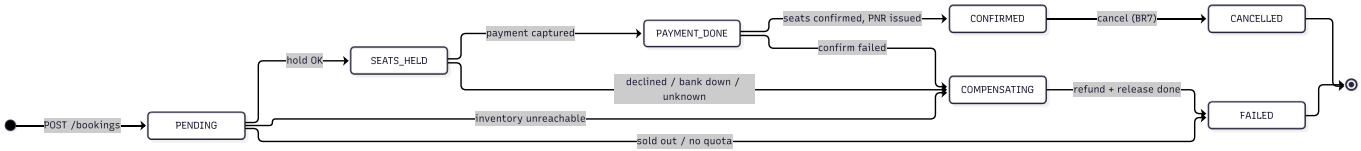
3.3 Main transaction - failure at step 3



The same transaction when the seat confirmation fails after the money was taken. booking-service compensates newest first: refund (undo step 2), then release (undo step 1).

This is the “money debited, ticket not booked” case, and here it ends correctly within the same request: the passenger gets `502 SEAT_CONFIRMATION_FAILED` with `booking_status: FAILED`, the payment is `REFUNDED`, the seats are back in the quota, and a refund SMS follows. Other failure paths use the same mechanism with a shorter plan: sold out at step 1 (nothing to undo, 409), bank declines at step 2 (release only, 402), bank unreachable (release only, 503), bank silent (refund-if-taken plus release, 504).

3.4 State machines

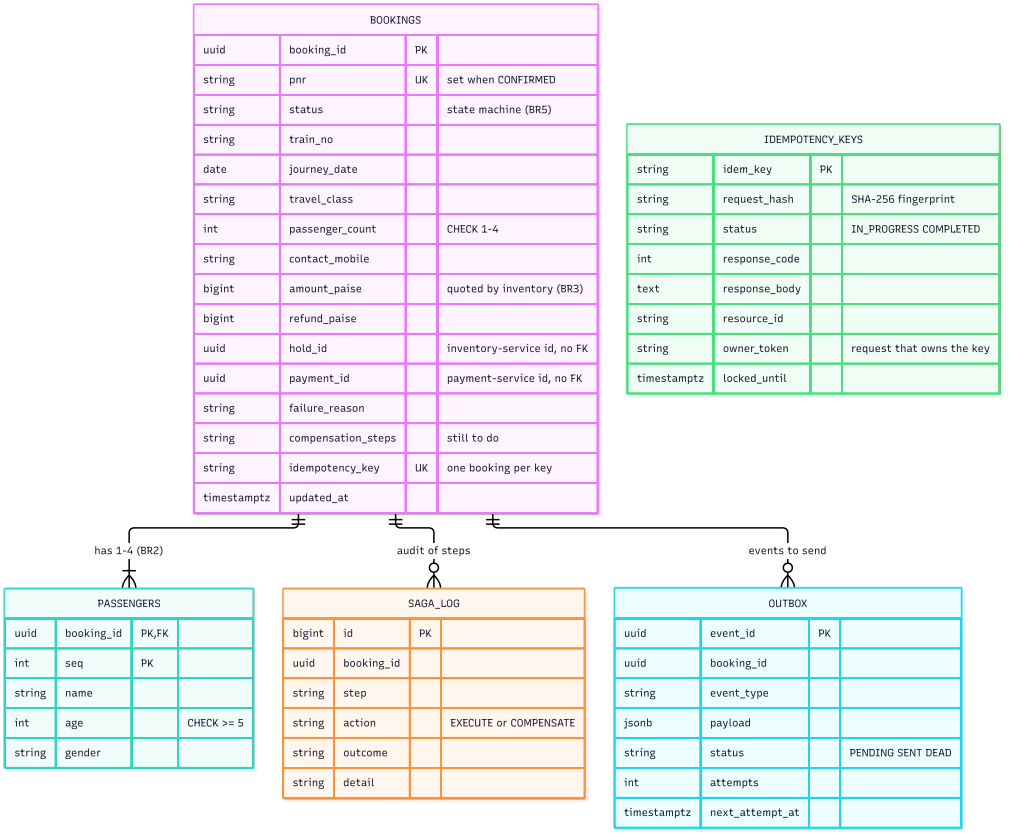


Booking states (BR5). booking-service checks every transition against this table and applies it with a conditional UPDATE, so two actors can never both move a booking.

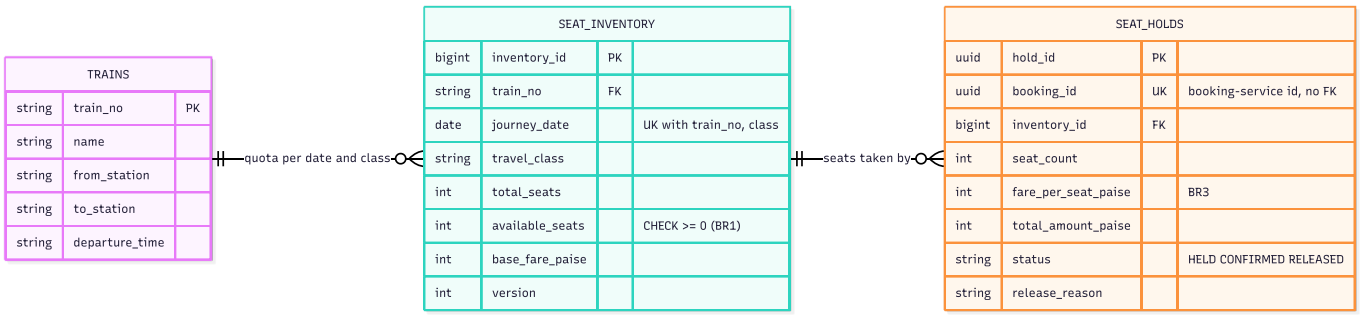
A seat hold goes HELD -> CONFIRMED -> RELEASED (or HELD -> RELEASED, also after a 10-minute expiry), and a payment PENDING -> CAPTURED -> REFUNDED (or PENDING -> DECLINED / FAILED / VOIDED); both diagrams are in `docs/diagrams/`. The hold is what makes the saga safe for other passengers: while a booking is between steps 1 and 3 its seats are HELD, so nobody else can sell them (a semantic lock).

4. Data design

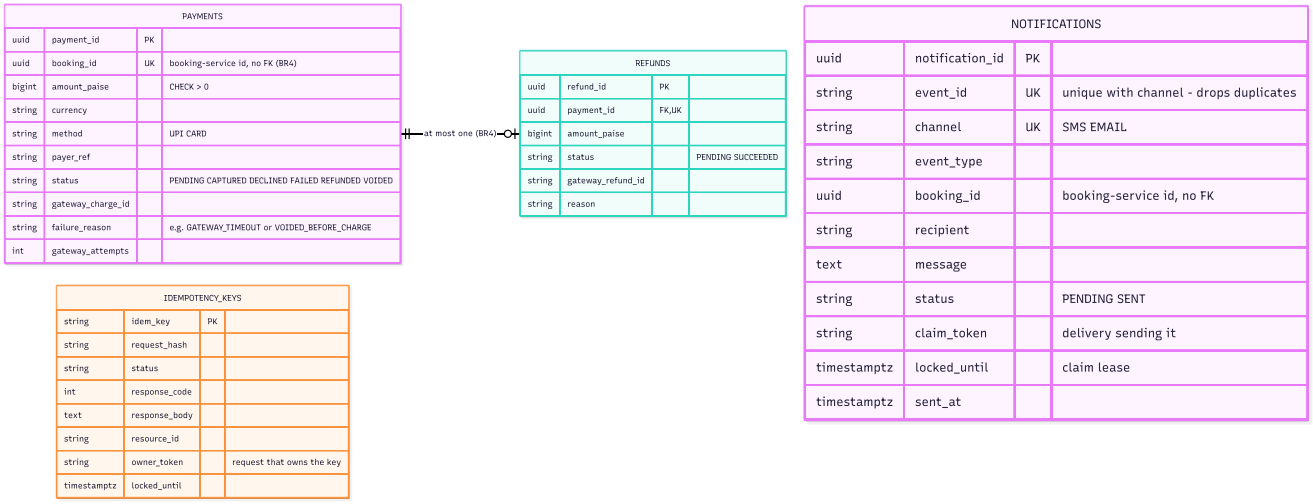
4.1 One database per service



booking_db: bookings and passengers, the saga log, the outbox and the stored idempotency keys.



inventory_db: trains, the Tatkal quota per train/date/class, and the seat holds.



payment_db (left) and notification_db (right). Foreign keys exist only inside one database; the ids of other services' entities are plain columns.

4.2 Data ownership

Data	Owner (only writer)	Other services keep	How they get it
Trains, fares, quota counters, seat holds	inventory	booking: `hold_id` and the quoted `amount_paise`	response of `POST /holds`; `GET /holds?booking_id=`
Bookings, passengers, saga log, outbox	booking	inventory, payment and notification: `booking_id`	`GET /bookings/{id}`
Payments, refunds	payment	booking: `payment_id`, `refund_paise`	response of `POST /payments`; `GET /payments?booking_id=`
Messages sent	notification	-	`GET /notifications?booking_id=`

4.3 Cross-service references

There are no foreign keys or joins across services: a service keeps only the ids of other services' entities and asks the owner when it needs more. Integrity across services is kept by the saga rather than by the database, helped by uniqueness on the referencing side - `UNIQUE (booking_id)` in `seat_holds` and in `payments` guarantees one hold and one payment per booking even if a step is retried. Booking keeps two deliberate copies: the amount quoted at hold time (the price the passenger agreed to, even if the fare changes later) and the refunded amount.

The four databases are physically separate containers with their own users (CS07 contrasts physical and logical separation; physical costs a little more memory but makes the boundary impossible to cross by accident and maps directly onto one database per pod in Assignment 2). `scripts/check_db_isolation.py` tries every service's credentials against every database; only the 4 diagonal combinations connect (evidence: `evidence/database-per-service/`). Our evidence scripts read the four databases to *prove* outcomes, but correlate them by id in Python, exactly as a reporting job would - no service does this.

5. API design

5.1 Conventions

All paths are versioned (`/api/v1/...`), bodies are JSON with snake_case names, ids are UUIDs, money is integer paise and dates are ISO 8601. Request models reject unknown fields, so a typo such as `passanger` fails loudly. Status codes: 201 created, 200 read or replay, 400 bad header, 402 payment declined, 404 unknown resource, 409 conflict with current state, 422 invalid request or broken business rule, 502/503/504 when a downstream step failed (with the booking's final status in the body). Every service serves Swagger UI at `/docs`; exported OpenAPI 3.1 files are in `docs/openapi/`.

5.2 Endpoints

Service	Method and path	Purpose	Main responses
booking	`POST /api/v1/bookings` (Idempotency-Key)	run the booking saga	201, 402, 404, 409, 422, 502, 503, 504
booking	`GET /api/v1/bookings`, `GET /api/v1/bookings/{id}`	list (filters) / read	200, 404
booking	`GET /api/v1/bookings/{id}/saga`	every step and compensation	200, 404
booking	`POST /api/v1/bookings/{id}/cancel`	cancel (BR7), safe to repeat	200, 404, 409, 503
inventory	`GET /api/v1/trains`, `GET /api/v1/trains/{no}/availability`	catalogue, seats left and fares	200, 404
inventory	`POST /api/v1/inventory`, `GET /api/v1/inventory`	open a Tatkal quota / read counters	201, 404, 409
inventory	`POST /api/v1/holds`	step 1, idempotent per booking_id	201 new, 200 existing, 404, 409
inventory	`POST /api/v1/holds/{id}/confirm`, `/release`	step 3 / compensation	200, 404, 409
payment	`POST /api/v1/payments` (Idempotency-Key)	step 2	201, 402, 409, 503, 504
payment	`POST /api/v1/payments/{id}/refund`	compensation, at most once	200, 409, 503
payment	`POST /api/v1/payments/void`	compensation when no payment exists yet	200
payment	`GET /api/v1/payments?booking_id=`, `/{id}`	read	200, 404
notification	`POST /api/v1/notifications`	deliver an event, duplicates ignored	201, 200 duplicate, 422, 503
all	`GET /health`	liveness + database check	200, 503

5.3 Sample request and responses

```
POST /api/v1/bookings
Idempotency-Key: 8d3c1f52-2b7e-4f0a-9c61-3e5a7d9b1c04
X-Correlation-ID: report-sample-01

{"train_no": "12951", "journey_date": "2026-11-05", "travel_class": "3A",
 "passengers": [{"name": "Asha Rao", "age": 34, "gender": "F"},
                {"name": "Vikram Rao", "age": 36, "gender": "M"}],
 "contact": {"mobile": "9876543210", "email": "asha@example.com"},
 "payment": {"method": "UPI", "payer_ref": "asha@okmock"}}
```

```
HTTP/1.1 201 Created
X-Correlation-ID: report-sample-01

{"booking_id": "90756b9d-7a11-4110-a581-fa7dd2cb267f",
 "pnr": "2441510525", "status": "CONFIRMED",
 "train_no": "12951", "journey_date": "2026-11-05", "travel_class": "3A",
 "passengers": [...], "contact": {...},
 "amount_paise": 579000, "refund_paise": null,
 "hold_id": "035b930b-b3d7-4c0b-be8f-4657269247b7",
 "payment_id": "8dd11b8b-6348-47f8-8ff3-d5b128a06080",
 "failure_reason": null, "created_at": "2026-10-21T07:17:29.472685Z", "updated_at": "..."}
```

The amount is never sent by the client: 2 x (Rs 2,495 base + Rs 400 Tatkal charge, 30 % capped for 3A) = Rs 5,790, quoted by inventory (BR3). The next booking for that run, with the quota now empty:

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{"type": "https://tatkal.example/problems/seats-unavailable",
 "title": "Not enough seats", "status": 409,
 "detail": "Requested 1 seat(s) in 3A on train 12951 for 2026-11-05, only 0 left",
 "instance": "/api/v1/bookings", "code": "SEATS_UNAVAILABLE",
 "booking_id": "e8006af3-eaf2-42b2-b21a-78e6c3bcf819", "booking_status": "FAILED",
 "correlation_id": "d7fc73c7090344e387be3ad4884f22bc"}
```

5.4 Error model

Every error from every service uses one format, RFC 9457 *problem details* (`application/problem+json`), extended with a stable machine-readable `code` and the request's `correlation_id`; business-rule failures add `rule`, and validation failures list field `errors`:

```
{"type": "https://tatkal.example/problems/passenger-under-minimum-age",
 "title": "Business rule violated", "status": 422,
 "detail": "Passenger 1 is 3. Children under 5 travel without a berth and are not booked ...",
 "instance": "/api/v1/bookings", "code": "PASSENGER_UNDER_MINIMUM_AGE", "rule": "BR2",
 "correlation_id": "1495c935d1ac4b30b57fab1aef6e07ee"}
```

We chose a published standard over an invented envelope so that clients and tools already understand it, and the `code` field gives scripts something stable to assert on. The handlers live in each service's `core/errors.py`; the OpenAPI files document every 4xx/5xx with this schema.

6. Engineering challenges

6.1 Challenge 1 - Concurrency control

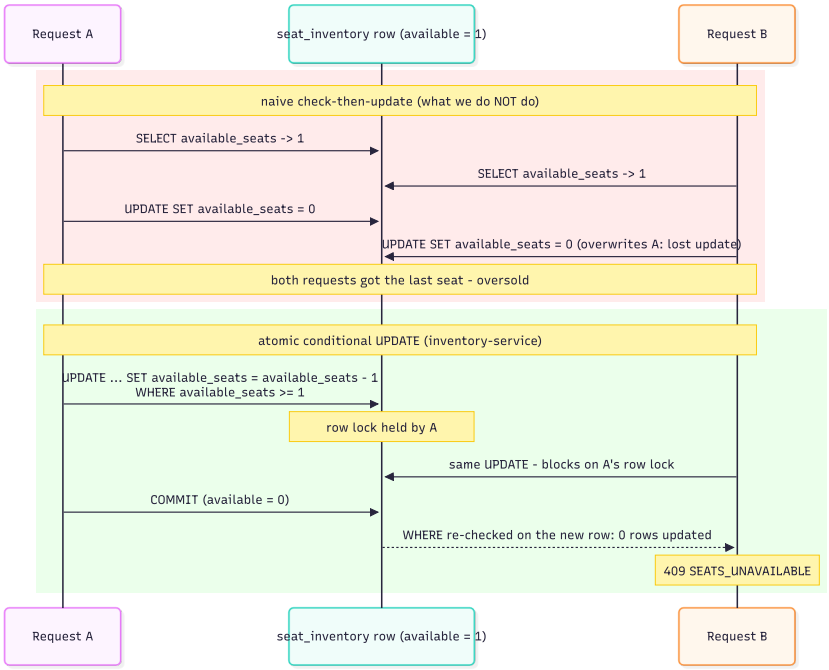
Problem. Twenty passengers press "Book" for the last five Sleeper seats in the same instant. The quota must never go below zero and exactly five must get a ticket.

Design. inventory-service takes seats with one atomic conditional update, and inserts the hold in the same transaction:

```
UPDATE seat_inventory SET available_seats = available_seats - :n, version = version + 1
 WHERE inventory_id = :id AND available_seats >= :n
RETURNING available_seats
```

PostgreSQL row-locks the quota row for the update; a concurrent transaction waits for the lock and then re-evaluates the `WHERE` clause against the committed row, so once the seats are gone the update matches nothing and the request gets `409 SEATS_UNAVAILABLE`. Two constraints back it up: `CHECK (available_seats >= 0)` and `UNIQUE (booking_id)` on holds (a retried hold cannot take seats twice).

Why plain check-then-update fails. Reading the counter, checking it in Python and writing back `value - n` are separate steps. Every request that reads before another writes sees the same free seat; under READ COMMITTED the second write simply overwrites the first (a lost update). More holds are created than seats exist, and even the CHECK constraint does not notice because the counter never goes negative. We kept that naive version behind a dev-only switch (`HOLD_STRATEGY=naive`) to prove the point.



Why check-then-update oversells and the conditional UPDATE does not.

Alternatives. `SELECT ... FOR UPDATE` then update is equally correct but holds the lock across a round trip from Python. Optimistic locking on `version` is correct too, but in a rush most requests would fail the version check and retry even while seats remain. One row per seat with `SKIP LOCKED` is the right model if passengers choose seats; we sell counts, so a counter is simpler. SERIALIZABLE isolation would work but turns the rush into serialization failures the client must retry.

Evidence (`evidence/challenge-1-concurrency/`). `challenge1_concurrency.py --strategy both` fires 20 bookings at a fresh 5-seat quota through the full saga. Naive: **20 of 20 confirmed**, 20 passengers charged Rs 22,100 for seats worth Rs 5,525, counter left at 4 with 20 holds. Atomic: **5 confirmed, 15 rejected with 409**, `available_seats = 0`, 5 confirmed holds, 5 captured payments, 15 failed bookings with no payment - all checks pass.

Limitations. The quota row is a hot row: every booking for one train/date/class queues on its lock (a few milliseconds each). That is fine for hundreds of concurrent requests; at Tatkal-day scale we would add a waiting room in front, or split the quota into buckets.

6.2 Challenge 2 - Idempotency

Problem. Phones lose connections, users tap twice and proxies retry. A repeated `POST /bookings` must not book or charge twice, and must give the client the same answer it missed.

Design (`services/idempotency.py`, the same in booking and payment).

1. The client sends an `Idempotency-Key`. A missing key is `400`.
2. We store the key, a SHA-256 fingerprint of method, path and the canonical JSON body, and a lease. The key is claimed with `INSERT ... ON CONFLICT DO NOTHING`, so the primary key lets exactly one of any number of simultaneous requests become the owner.
3. The owner runs the saga and stores the status code and the exact response bytes.
4. Every other request with the key gets one of three answers:
   - a different fingerprint gets `422 IDEMPOTENCY_KEY_REUSED`;
   - if the key is completed, the stored response is replayed with `Idempotent-Replayed: true`;
   - if the key is still in progress, the request waits for it and then replays the stored response.
5. If the owner dies, its lease expires and the next request takes over, reusing any booking already made with the key (UNIQUE on bookings), so a takeover never starts a second saga. An owner token stops the old owner overwriting the result. Keys are purged after 24 hours.

Behind this are two more layers: `UNIQUE (booking_id)` on payments refuses a second charge even with a new key (409, BR4), and every call to the bank carries the same gateway key, so a retry after a timeout is deduplicated by the bank itself. Hold, confirm, release, refund and cancel are idempotent by state and need no key.

Alternatives. A client-chosen booking id with `PUT /bookings/{id}` gives natural idempotency, but it would leak id generation to clients and still needs fingerprinting. A Redis key store is faster but adds a component and loses the transactional link to the booking database. Stripe-style "409 while in progress" is simpler; we wait instead so duplicates get the real result (bounded by 20 s, then 409).

Evidence (`evidence/challenge-2-idempotency/`, 14 checks pass). Three sequential repeats return byte-identical responses, the 2nd and 3rd marked as replays. A declined (402) booking is replayed too, and the bank's request counter does not move on the repeat. The same key with one passenger instead of two gets 422; no key gets 400. **Ten identical requests released together** produce one booking, one payment and one capture at the bank: one original response and nine replays, all byte-identical.

Limitations. A waiting duplicate holds a worker thread while polling every 100 ms. Keys are global rather than per user (no authentication). After 24 hours a repeat is answered from the booking itself, not from the stored bytes.

6.3 Challenge 3C - Saga with compensation

Problem. A booking changes three databases. A distributed transaction (2PC) is not an option: it needs every participant up at the same time, holds locks across services and is not available across independent PostgreSQL instances behind REST (CS07). We need consistency without it.

Design. An orchestrated saga in booking-service with explicit states (section 3.4):

Step	Service	Kind	Compensation
1 HOLD_SEATS	inventory	compensatable	RELEASE_SEATS
2 CHARGE_PAYMENT	payment	compensatable	REFUND_PAYMENT
3 CONFIRM_SEATS	inventory	pivot - after it the booking stands	none
4 NOTIFY	notification via outbox	retriable	none, retried until delivered

Each state change is committed before the next remote call and each step and compensation is appended to `saga_log`, so the database always shows how far a booking got. On failure the pure function `compensation_plan()` (BR6) decides what to undo, newest first, counting anything that *may* have happened - after a timeout we refund "if taken" and release "if held", both harmless when there is nothing to undo. Compensations are idempotent (a payment refunds at most once, a hold releases at most once, a 404 means nothing to undo) and are all attempted even if one fails, so a bank outage does not keep seats hostage. Steps that still fail stay in `compensation_steps`; a recovery thread retries them, and also compensates sagas abandoned by a crash (no progress for 60 s).

Forcing the failure. inventory-service has a dev-only fault injector (`POST /api/v1/admin/faults {"operation": "confirm_hold", "times": 2}`); two failures because booking retries an idempotent call once.

Evidence (`evidence/challenge-3c-saga/`, 11 checks pass). Forced failure at step 3: `502`, saga log `HOLD ok, CHARGE ok, CONFIRM failed, REFUND ok, RELEASE ok`, booking FAILED with Rs 9,800 refunded, payment REFUNDED, hold RELEASED, quota restored, refund SMS sent. The whole request is traced through all five services' logs by one correlation id. Failure at step 2 (declined): only RELEASE runs. A failed compensation: with the bank's refund API down, the booking stays COMPENSATING (seats already released); when the bank recovers, the recovery job completes the refund with no API call, 21 s later. A final check reads all four databases and finds every booking consistent.

Alternatives. Choreography would need a broker and spread the flow over four services; with one clear owner of the booking lifecycle, orchestration is easier to follow and test (section 8). Forward recovery (retry step 3 until it works) is friendlier, but a confirm can fail for good, so we compensate backwards. The hold/confirm/release API is effectively Try-Confirm-Cancel on the inventory side.

Overlaps. If booking gives up on a charge that is still running, its refund gets 503 `PAYMENT_IN_PROGRESS` until the charge settles; if no payment exists yet, booking voids it (`POST /payments/void`) so the late charge is refused. A

charge stops retrying once its payment is closed, and a capture that lands on a voided payment is refunded at once. Holds still HELD after 10 minutes are released as a last safety net.

Limitations. Seats of a booking that ends up compensated are held for its duration (usually well under a second, a few seconds more when the bank is slow). A saga quiet for 60 s is assumed dead; timeouts keep real sagas well below that. We trust the bank's lookup by key to include requests still in flight, as the mock bank does; with a real bank the settlement report would be the final check.

6.4 The slow or failing bank (condition e)

Although 3A was not our chosen challenge, payment-service needed basic resilience to make the saga honest. Every bank call has a 2 s timeout and up to three attempts with exponential backoff and full jitter (0-200 ms, then 0-400 ms), always with the same gateway key. A connection refused, or 503 on every attempt, means nothing was taken (FAILED, 503). A request that was sent but never answered means the outcome is unknown: the payment stays PENDING and returns 504. A PENDING payment is reconciled with the bank before any refund, once its charge can no longer be running.

`scripts/bank_failure_modes.py` (evidence in `evidence/bank-failures/`) books through the normal API under each bank behaviour:

- **Slow bank (3 s).** The first attempt times out and the retry gets the bank's stored result. The booking is CONFIRMED after 3.1 s with two attempts and exactly one capture.
- **Bank down.** The booking fails in 0.4 s with 503, the payment is FAILED and the seats are released.
- **Flaky bank (60 % of calls fail).** Three of four bookings succeeded (one on a retry) and one failed cleanly after three attempts with its seats released; none was captured twice.

7. Testing

Strategy. Most tests are cheap unit tests of the business rules, and fewer, slower tests run against real infrastructure:

- **Unit tests** (94) test every rule BR1-BR7 as plain functions, plus the fingerprint, backoff, retry classification (with `httpx.MockTransport`, no network) and message templates.
- **Integration tests** (30) run against the service's real PostgreSQL, in a throwaway schema per run: 20 threads racing for 5 seats (and the naive version overselling), parallel same-key and same-booking payments, a hold retry overlapping the first attempt, hold expiry, refunds and voids racing a charge, key takeover, duplicate and partly failed notifications, and the saga with fake inventory and payment services (step-3 failure order, failed compensation finished by recovery, abandoned saga, unknown hold).
- **mock-bank-gateway** has 8 component tests, including 10 concurrent same-key charges.
- **End-to-end** (7, `tests/e2e/`) drive the main transaction through the public APIs: book, pay, confirm and notify (and the correlation id appears in four services' logs), plus the step-3 failure, a declined payment, a sold-out train, a cancellation and BR2 violations.
- **Challenge scripts** are scenario tests that end with database checks.

Results. All 132 service tests and 7 end-to-end tests pass (`evidence/tests/`); all three challenge scripts pass on fresh databases.

Service	Unit	Integration	Rules covered
inventory-service	23	4	BR1, BR3
payment-service	31	10	BR4
notification-service	10	5	(duplicate events)
booking-service	30	11	BR2, BR5, BR6, BR7
mock-bank-gateway	8 (component)	-	-

Not covered: load beyond about 30 concurrent requests, several instances of one service (Assignment 2), and network partitions between a service and its database.

8. Design decisions and trade-offs

Decision	Options	Chosen and why	Cost we accept
Communication for the booking steps	sync REST; async messaging over a broker	**sync REST**: the passenger waits for a yes or no, step 2 needs the price from step 1, and no broker is needed for Assignment 1	booking's availability depends on inventory and payment being up; mitigated by timeouts and compensation
Coordinating the saga	orchestration; choreography	**orchestration** in booking-service: the whole flow is in one file, states are explicit, and the saga log answers "what happened to my booking?"	booking knows the other APIs; more logic in one service
Notifications	direct call in the request; transactional outbox; broker	**outbox + dispatcher**: the event commits with the booking, delivery is retried, a dead SMS provider never fails a booking; the consumer drops duplicates by `event_id`	a few seconds of delay; at-least-once delivery that needs an idempotent consumer
Preventing overselling	check-then-update; `FOR UPDATE` ; optimistic version; conditional UPDATE	**conditional UPDATE**: one statement, no retry loop, correct under READ COMMITTED	a hot row per train/date/class
Database separation	one server with 4 schemas or databases; 4 servers	**4 containers, 4 users**: the boundary cannot be crossed by accident, matches one database per pod later	more memory; no cross-service reporting by SQL
Shared code	shared library; copy small plumbing	**copy** the ~370-line `core/` package (logging, errors, middleware, health): each service builds and deploys alone (CS05 warns that shared libraries couple services)	the same fix must be applied five times
Failure recovery	forward (retry to completion); backward (compensate)	**backward**: always possible and leaves nothing half done	a passenger whose step 3 failed must book again

9. Limitations and future work

- **Scale.** We tested one instance of each service. The background jobs already use `FOR UPDATE SKIP LOCKED`, and idempotency lives in the database, so they should work with replicas (Assignment 2). The fault injector and the mock bank keep their state in memory per process.
- **The rush itself.** A real Tatkal opening needs admission control - a virtual waiting room or rate limiting in front of booking - and a way past the single hot quota row.
- **Resilience.** There is no circuit breaker around the bank yet (the next step after timeouts and retries), holds expire only through a coarse 10-minute sweep rather than a real reservation timer (3B), and notifications could move to a broker (3D).
- **Operations.** Tables are created at start-up; a real system needs migrations (Alembic). There are no metrics or traces yet beyond correlated logs. Dev passwords live in `.env` files and will move to Kubernetes secrets in Assignment 2. Seat allocation, waiting list, RAC and charting are also missing.

10. Links

Source code: https://github.com/spaceman-dev/tatkal-booking-microservices
Google Drive (video, report, documents): [add link before submission] - Demo video: [add link]

Appendix A - Contribution table

Member	BITS ID	Owned	Other contributions	Share
Dhruv Bhat	[BITS ID]	booking-service: saga orchestration, idempotency store, outbox dispatcher, recovery job	repository and tooling (`stack.py`), integration of all services, report sections 2, 3 and 8	25 %
Ratani Snehi Rajesh Darshana	[BITS ID]	inventory-service: quota, Tatkal fares (BR3), atomic holds (BR1), fault injection	Challenge 1 script and evidence, ER diagrams, report sections 4 and 6.1	25 %
Atharva Kadam	[BITS ID]	payment-service and mock-bank-gateway: idempotent charges and refunds (BR4), retries with backoff	Challenge 2 script and evidence, OpenAPI export, report sections 5 and 6.2	25 %
N Abhishek Reddy	[BITS ID]	notification-service: templates, duplicate-safe delivery	end-to-end tests, Challenge 3 script and evidence, report sections 6.3, 7 and 9, video editing	25 %

Appendix B - Individual reflections

Dhruv Bhat (booking-service). I built the booking saga, the idempotency store and the two background jobs. The hardest problem was deciding what a failure *means* to the saga. A decline from the bank is a clear “no”, a refused connection means nothing happened, but a timeout after the request was sent means the money may or may not have been taken - so the compensation plan works with “may have happened” and every compensation has to be safe when there is nothing to undo. Two things surprised me while testing. Our first forced failure at step 3 did not fail at all, because my client correctly retried the idempotent confirm call once and the retry succeeded - the fault had to cover both attempts. And a booking left over from a crashed test run was retried by the recovery job every 20 seconds because inventory answered 404 for its hold; a 404 has to count as “nothing to undo”, and the tests should never have shared tables with the running service. If I started again I would make the saga steps a small table-driven engine instead of one long method, and replace the polling wait for duplicate idempotency keys with PostgreSQL `LISTEN/NOTIFY`.

Ratani Snehi Rajesh Darshana (inventory-service). I owned the seat quota, the fare rule and the hold API. The hardest part was proving the concurrency control rather than just writing it. A check-then-update looked correct in every single-request test, so we kept a naive version behind a dev switch and raced 20 bookings against it: it sold all 20 seats of a 5-seat quota and left the counter at 4. Seeing that number made it obvious why the check and the decrement must be one statement, and why the CHECK constraint alone could never catch a lost update. I also learned that “idempotent per booking_id” needs more than a unique constraint: a review found that a retry overlapping the first hold request got 409 and left that hold’s seats owned by nobody. Inserting the hold row before taking the seats fixed it, and we added a 10-minute expiry for holds as a safety net. With more time I would split the hot quota row into buckets.

Atharva Kadam (payment-service and mock-bank-gateway). I built the payment service, its client for the bank and the mock bank itself. The hardest problem was the difference between “the bank said no” and “the bank did not answer”. A declined payment is final, a refused connection means nothing happened, but a timeout after the request was sent is unknown. I handled it by keeping such payments PENDING, retrying only with the same idempotency key, and reconciling with the bank before any refund. Making the mock bank deduplicate requests while the first one is still being processed is what lets the slow-bank run end with exactly one capture over two attempts. A review also found a gap I had missed: a refund arriving while my charge was still retrying voided the payment, and a capture that came back afterwards was kept. Closing it took a “still wanted?” check before every attempt, a 503 while a charge may still be running, and an immediate refund for a capture that lands on a voided

payment. Next I would add a circuit breaker so a dead bank fails fast instead of costing three attempts per request.

N Abhishek Reddy (notification-service and end-to-end tests). I owned notifications and the end-to-end and saga test scripts. The interesting part of notifications was that the outbox delivers *at least* once, so my service had to be an idempotent consumer: it claims each channel of an event with a unique row, marks it SENT as soon as it goes out, and on a provider failure gives back only what was not sent. My first version released the whole event when only the e-mail failed, so the retry sent the SMS twice, and it answered “duplicate” while another delivery was still sending. The hardest problem on the testing side was making failure scenarios deterministic. Our first attempt at “the refund fails” depended on timing; adding a partial-outage mode to the mock bank (refunds down, charges up) made it repeatable. Next time I would add a dead-letter view for events that can never be delivered, and run the e2e suite in CI on every push.

Appendix C - AI-use declaration

We used **GitHub Copilot** in VS Code, in agent mode, for this assignment. As the course policy asks, here is what it was used for:

- **Design.** It proposed the Tatkal domain, the four-service decomposition, business rules BR1-BR7 and the choice of Challenge 3C, working from the assignment brief and our lecture slides and lab sheets.
- **Code.** It wrote the code of the five services, the unit, integration and end-to-end tests, the run and challenge scripts, and the Mermaid sources of the diagrams.
- **Running, evidence and review.** It ran the test suites and the scripts that produced the files in `evidence/` (the outputs are unedited). A separate Copilot review agent then reviewed the code; the six problems it found (mostly retries, compensations and late results racing each other) were fixed and retested.
- **Writing.** It drafted the README files, this report (including first drafts of the reflections in Appendix B) and the video script.

[To be completed by the team before submission: what each member reviewed, changed or rewrote, and how the team checked the work.]

Appendix D - Business rules traceability

Rule	Code	Tests
BR1	`inventory-service/app/repository/inventory_repo.py` (`decrement_if_available`), `services/holds.py`, `domain/allocation.py`	`tests/unit/test_br1_allocation.py`, `tests/integration/test_br1_concurrent_holds.py`, `test_hold_retry_and_expiry.py`, challenge 1
BR2	`booking-service/app/domain/booking_rules.py` (`validate_passengers`)	`tests/unit/test_br2_passengers.py`, e2e
BR3	`inventory-service/app/domain/fares.py` (`tatkal_charge_paise`, `quote`)	`tests/unit/test_br3_fares.py`, e2e
BR4	`payment-service/app/domain/payment_rules.py`, `services/payments.py`, `UNIQUE(booking_id)`	`tests/unit/test_br4_payment_rules.py`, `tests/integration/test_idempotent_payments_db.py`, `test_payment_fencing_db.py`, challenge 2
BR5	`booking-service/app/domain/booking_rules.py` (`TRANSITIONS`), `repository/bookings_repo.py` (`transition`)	`tests/unit/test_br5_state_machine.py`
BR6	`booking_rules.py` (`compensation_plan`), `services/saga.py`, `services/recovery.py`	`tests/unit/test_br6_compensation_plan.py`, `tests/integration/test_saga_db.py`, e2e, challenge 3
BR7	`booking_rules.py` (`tatkal_cancellation`), `services/saga.py` (`cancel`)	`tests/unit/test_br7_cancellation.py`, e2e